

Tecnológico de Monterrey

Campus Santa Fe

Arquitectura de SIRAI y justificación de la base de datos

Integración de seguridad informática en redes y sistemas de software

Oscar Miguel Lara Elizondo

Jesús José Espinoza Torruco

Juan Pablo Briñez Corzo

Adolfo Hernández Sánchez

A01781855

A01781963

A01782050

A01782496

21 de octubre

Índice

Índice **1**

1. Arquitectura SIRAI **2**

1.1 Funcionamiento de cada elemento 3

1.2 Flujo de las peticiones 4

1.3 Decisiones de seguridad 5

1.4 ¿Por qué este stack? 6

2. Justificación MySQL **6**

2.1 Relación de los datos 6

2.2 Reglas de la base de datos 7

2.3 Transacciones en la plataforma 7

2.4 Bitácora sin posibilidad de cambio o eliminación 8

2.5 Índices pensados para las consultas 8

3. Normalización de la base de datos **9**

3.1 Normalización a tres formas normales 9

3.2 Justificación de decisiones de diseño 9

4. Diagrama entidad-relación **11**

5. Infraestructura **12**

5.1 Resultados 13

5.2 Interpretación de resultados 13

5.3 Uso de disco 14

5.4 ¿Qué se necesita para el programa? 15

5.4 Límites de la medición 15

6. Diferencias de nuestro SIRAI y el SRS **15**

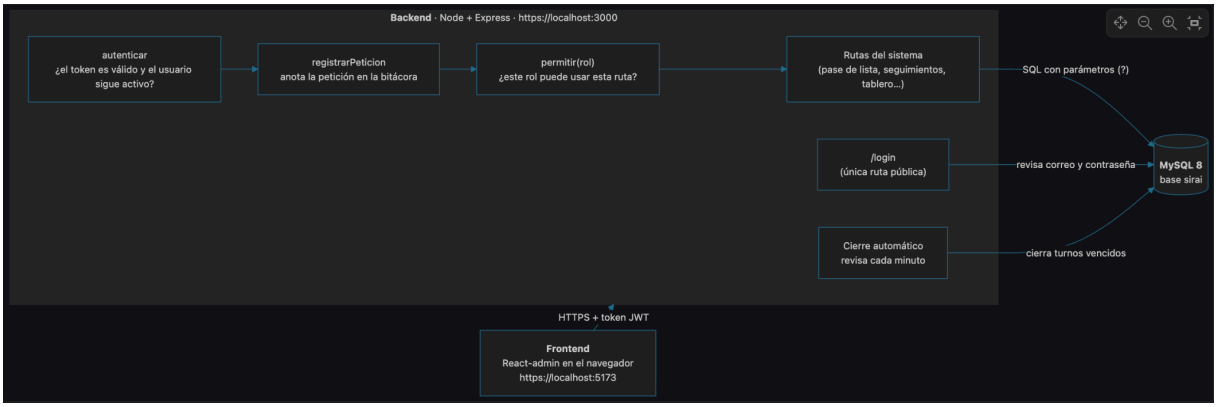
Arquitectura de SIRAI y justificación de la base de datos

El documento explica cómo fue construido SIRAI y la justificación del diseño propuesto para la base de datos.

1. Arquitectura SIRAI

Sirai contiene tres piezas claves en donde siguen un proceso ordenado de comunicación. Estas tres piezas:

- La pantalla que usa el usuario
- El servidor que decide qué acciones se pueden llegar a realizar
- La base de datos que mantiene la información de la plataforma



1.1 Funcionamiento de cada elemento

Frontend

- Es la interfaz visual que el usuario ve e interactúa. Ejemplos de pantallas dentro de SIRAI son: login, menús, pantallas correspondientes a su rol. Este elemento no decide permisos, sino que únicamente muestra lo esencial para el usuario.
- El frontend de SIRAI fue realizado con React-admin, utilizando Vite para el servicio vía HTTPS.

Backend

- Es el elemento que realiza el procesamiento de las peticiones, revisa los permisos de los usuarios, aplica las reglas de la plataforma, habla con la base de datos y realiza la lógica de toda la plataforma.
- El backend de SIRIA utiliza las tecnologías de Node js utilizando Express.

Base de datos

- Es el elemento que guarda los datos y estipula reglas que no deben aunque en el backend pueda surgir un error.
- La base de datos de SIRIA utiliza MySQL 8, utilizando un motor InnoDB y caracteres utf8mb4.

1.2 Flujo de las peticiones

El flujo de las peticiones cambia respecto al rol del usuario y la acción a realizar. Pero en general, el flujo es el siguiente:

1. **Iniciar sesión:** El usuario escribe su correo y contraseña. El backend compara la contraseña con el hash utilizado (argon2id) guardado. Si el usuario se llega a equivocar un total de 5 veces, la cuenta sufre un bloqueo de 15 minutos. Si el usuario ingresa las credenciales correctas, el backend entrega un token JWT que dura un total de 8 horas. Este token se guarda en el front en el sessionStorage, la cual se borra al cerrar la pestaña.
2. **Mandar la petición:** Al mandar la petición, el front llama a su respectivo endpoint por vía HTTPS y manda consigo el token.
3. **Autenticar:** Se revisa que el token proporcionado sea válido y este no se haya vencido. Además de este proceso, se consulta en la base de datos que el usuario tiene un estado de activo, con el mismo rol y el respectivo requerimiento que se necesita.

Para la petición. Si uno de estos es incorrecto, la plataforma responde con un código de estado 401 y la sesión se cierra.

4. **Registrar la petición:** Se anota la petición en la bitácora cuando esta finaliza con su respectivo código de estado. Todas las rutas quedan registradas para tener un control sobre quién realizó la respectiva petición. La bitácora es una auditoría de los logs.
5. **Verificación de permisos:** Se revisa que el respectivo rol pueda realizar la respectiva acción. Si esto es incorrecto, se responde con un código de estado 403, que señala que los roles salen de sus respectivos permisos de visualización ya estipulados previamente.
6. **Validación y prevención de vulnerabilidades:** Si el permiso es correcto, la petición es aceptada. Se protegen las estancias al ser tomadas del token y no de lo proporcionado por el navegador. Las consultas SQL usan parámetros ?, evitando así las inyecciones SQL. Todos los datos sensibles se cifran antes de guardarse en la base de datos.
7. **Revisión de reglas y guardado:** La base revisa las reglas estipuladas y guarda la petición realizada. Si la operación contiene varios pasos, se guarda todo lo ocurrido o no se guarda nada si la petición es inválida.

1.3 Decisiones de seguridad

- Toda la comunicación dentro de la plataforma viaja por HTTPS, tanto lo que sucede en el frontend como en el backend; esto se logra mediante los certificados (backend.key y backend.crt).
- Únicamente la ruta de login es pública. En index.js, todas las rutas restantes se montan debajo de un proceso de autenticación y de registro de peticiones.

- El backend es el que decide qué elementos puede llegar a visualizar el usuario. El frontend esconde estas rutas, pero si alguien llama directamente, el backend se protege con un código de estado 401 o 403.
- Toda la información sensible de los usuarios de la plataforma es cifrada con AES-256-GCM. Estos elementos cifrados son los teléfonos de los tutores y contactos de emergencia, notas de llamadas, detalles de la causa de una falta, observaciones del alta médica y la nota de justificación.
- La bitácora que contiene la auditoría de los logs no puede sufrir alteraciones. Cada registro guarda el hash anterior y dos triggers de MySQL impidiendo que se pueda modificar o borrar.

1.4 ¿Por qué este stack?

EL SRS proporcionado por el ISSTE proponía el uso de PHP con CodeIgniter 4 en un patrón MVC. Nosotros, por nuestra falta de conocimiento en estas tecnologías y el corto tiempo que teníamos para generar una propuesta decente en tiempo y forma, propusimos el uso de Node, Express, React-admin y MySQL, siendo el stack que aprendimos durante el curso.

2. Justificación MySQL

La principal razón por la cual nosotros decidimos utilizar MySQL como nuestra base de datos proviene de que los datos de SIRAI son relaciones; un caso es que un infante debe pertenecer a una estancia, una falta abre un caso, un caso tiene llamadas, etc. Además de que existen reglas que no se pueden llegar a romper. Con todo esto, además, se aclara dentro del SRS proporcionado el uso de una base de datos relacional; por ende, opciones que también vimos, como MongoDB, no fueron utilizadas.

2.1 Relación de los datos

La base de datos contiene un total de 18 tablas, 14 llaves primarias y 44 llaves foráneas. Esto describe la necesidad de contener una base de datos relacional, porque una llave foránea apunta a un registro de otra tabla. En una base de datos no relacional, estas reglas se tendrían que programar aparte por vía del backend, generando la posibilidad de realizar un error que genere el mal uso y funcionamiento de los datos.

2.2 Reglas de la base de datos

En la base de datos de SIRAI existen un total de 7 reglas estipuladas. Estas reglas son condiciones que la base revisa en cada ingreso o cambio de los datos, sin importar la persona que lo realice.

- **chk_usuarios_alcance:** Impide que un docente no tenga una estancia, o que un administrador central esté atado a una estancia. Verifica que cada rol tenga su respectivo alcance.
- **chk_salida_requiere_entrada:** Impide que se pueda registrar la salida de un infante que nunca llegó a ingresar.
- **chk_orden_horas:** Impide que se ingrese una hora de salida con tiempo previo al de entrada.
- **chk_cierre_manual_con_usuario:** Impide que se haga un cierre de lista de manera manual sin el registro del usuario que lo llegó a realizar.
- **chk_cierre_completo:** Impide que se marque un caso como cerrado sin la fecha de cierre y registro de quien lo realizó.
- **chk_clasificado_con_cause:** Impide que se cree un caso sin una causa; es necesario de este mismo para clasificar el caso con su estado.

- **chk_un_destinatario:** Impide que el registro de una llamada no tenga la información o contenga más de dos datos a la vez. Esta información debe ser de un tutor o de un contacto de emergencia.

2.3 Transacciones en la plataforma

Algunas operaciones dentro de la plataforma tienen varios pasos, los cuales no pueden quedarse a la mitad del proceso. Un ejemplo es el cierre de la pase de lista:

1. Se registra el cierre del turno.
2. Los infantes sin registro quedan como falta injustificada.
3. Cada falta abre un caso de seguimiento.

Los tres pasos pasan por una transacción, la cual verifica que, si alguno de los pasos llega a fallar, la base de datos deshace todo lo ocurrido. MySQL también tiene la propiedad de bloquear filas mientras realizan una operación. Así se protege por si llegan dos peticiones al mismo turno: uno espera a la otra y no se llegan a cruzar.

2.4 Bitácora sin posibilidad de cambio o eliminación

La bitácora es un elemento que guarda una auditoría de quién realizó cierta acción y cuándo. Existen esencialmente dos triggers que hacen que MySQL rechace cualquier intento de modificar o borrar un registro. Cada registro guarda el hash del anterior; para identificar si un usuario hace un cambio de un registro viejo, se romperia esta cadena de registros y se notaria.

2.5 Índices pensados para las consultas

Además de las llaves se implementaron indeices que se tienen pensado para las consultas mas frecuentes de la base de datos y agilizar el proceso:

- **ix_tablero (estancia_id, fecha, turno):** Muestra el tablero con el semáforo de asistencias y el pase de lista.
- **ix_bandeja (estado, fecha_probable_retorno):** Muestra la bandeja de casos abiertos.
- **ix_reintentos (siguiente_intento_en):** Muestra los reintentos de llamadas agendadas.
- **ix_audit_entidad, ix_audit_usuario:** Ayuda en la búsqueda dentro de la bitácora por registro o por usuario.

3. Normalización de la base de datos

3.1 Normalización a tres formas normales

Primera Forma Normal (1FN): Cada celda guarda un solo dato.

Ejemplo de implementación: Un infante puede tener varios tutores, en lugar de guardar todos sus tutores en una celda “Maria, Jesús, Pedro”, existe la tabla **infante-tutor** donde se guarda una pareja por fila “Niño-Tutor”, Lo mismo ocurre con el contacto de emergencia, donde existe una fila por contacto de emergencia con su prioridad.

Segunda Forma Normal (2FN): En una tabla con llave de varias columnas, cada dato depende de la llave completa.

Ejemplo de implementación: La llave primaria de *“infante_tutor”* es el compuesta, conformada por el par *“(infante_id + tutor_id)”*. En la tabla porque dependen de este mismo par de “infante-tutor” se guarda el *“parentesco”* y *“autorizado_recoger”*, la misma persona puede ser *“madre"* de un niño y *"tía"* de otro, y estar autorizada a recoger a uno y no al otro. El nombre o el teléfono del tutor, en cambio, dependen solo del tutor y por eso viven en `tutores`.

Tercera Forma Normal (3FN): ningún dato depende de otro dato que no sea la llave.

Ejemplo de implementación: La tabla “Asistencias” no guarda el nombre del estado de una *“Falta injustificada”*, sólo un código que apunta al catálogo de *“Estados de asistencia”*.

3.2 Justificación de decisiones de diseño

a) Catálogos en tabla en lugar de listas fijas.

Las tablas “Estados_Asistencia” y “Causas_Inasistencia” existen como tablas independientes y no como una lista fija dentro de una columna porque cada **estado** y cada **causa** tiene datos propios que cambian el comportamiento del sistema. Una causa trae el texto de asesoría para el personal del ISSSTE, ejemplos: si requiere alta médica y avisa a los contactos de emergencia. Ahora al trabajar como una tabla, cambiar “Qué estado abre un caso” es editar un renglón, no cambiar el esquema ni el código.

b) Asistencias repite la estancia a propósito.

En teoría la estancia de una asistencia se podría sacar del infante utilizando *“infantes.estancia_id”*. Aún así, *asistencias* tiene su propio registro de *estancia_id*. Las razones de guardar esto repetido es:

- Velocidad: El tablero y pase de lista busca “todas las asistencias de esta estancia, este día y este turno”. Con la estancia en la misma tabla, el índice *“ix_tablero”* encuentra esas filas directamente, sin tener que unir con *“infantes”*. Esta es una petición directa del SRS.
- Reglas escritas en la base: La llave *“fk_asistencia_turno”* (estancia_id, turno), obliga a que la asistencia use un turno que sea válido para esta estancia. Sin la estancia en la tabla, la regla no se podría escribir.
- El historial no cambia: Si un infante se cambia de estancia, sus asistencias anteriores siguen diciendo dónde ocurrieron.

c) Las justificaciones de inasistencias viven junto a las asistencias

La justificación de un “Ausente justificado” son columnas dentro de la tabla de “*asistencias”* y no una tabla aparte. Cada asistencia tiene como máximo una justificación y depende solo de esa asistencia, por lo que separarla en una tabla adicional no proporciona ninguna ventaja.

d) Borrado lógico, uso de “*deleted_at” y “*RESTRICT”

Casi todas las tablas tienen “deleted_at”. En un sistema que maneja información de niños, nada debe desaparecer sin rastro, una estancia desactivada conserva su historial de asistencia y se puede volver a activar. Por eso todas las Foreign Keys utilizan RESTRIC, para no dejar borrar algo que otros registros usan. No se utiliza CASCADE porque necesitamos utilizar CHECK, y no se puede utilizar sobre columnas con CASCADE o SET NULL, que utilizamos igual. Para “eliminar” filas de tablas, utilizamos el tag “deleted_at” para poner la fecha de cuando se eliminó y ser ignorado por las consultas.

e) La bitácora de logs “audit_log” no tienen FKs

“audit_log” es la única tabla sin llaves foráneas, porque tiene que sobrevivir a cualquier cambio realizado. Si un usuario se desactiva o un registro se borra, sus anotaciones siguen ahí. Y es precisamente esa la misma razón guarda el “antes” y “después” de los cambios en columnas “JSON”. Solo es como un “Snapshot” de lo que era y ahora es, para referencia, y no necesita estar ligado a alguna otra tabla.

f) Identificadores UUID

Las tablas con datos de personas y su actividad (usuarios, infantes, tutores, asistencias, seguimientos, llamadas, etc.) usan UUID v7 de 36 caracteres. Solo las regiones y estancias usan “INT AUTO_INCREMENT”.

g) Columnas cifradas

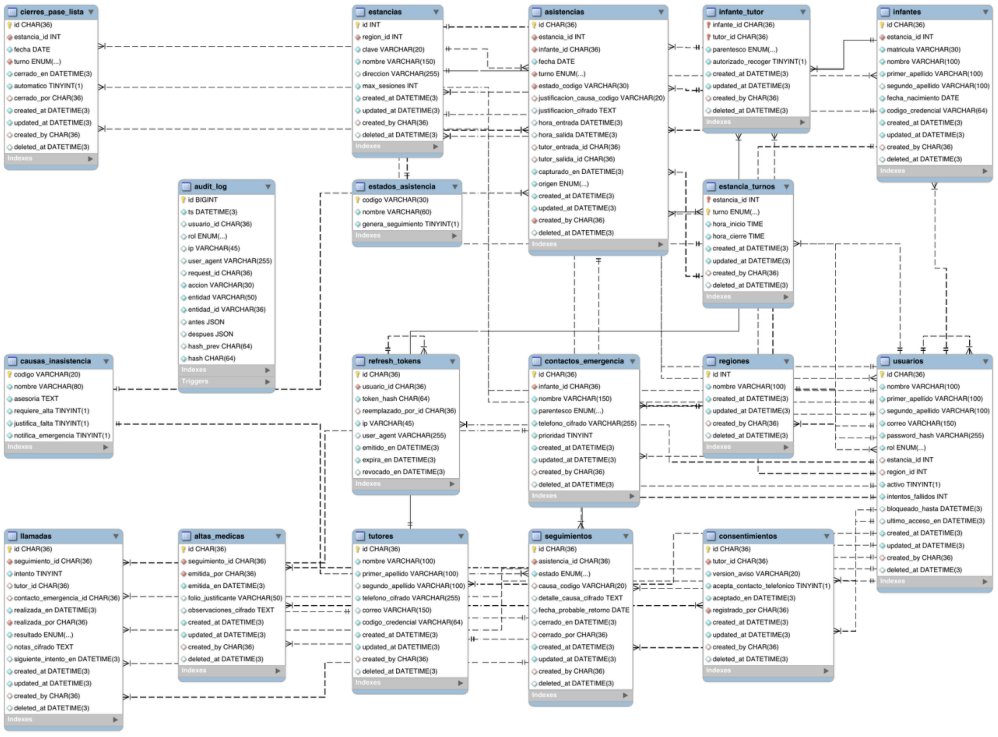
Las columnas que terminan en *“cifrado”* guardan el dato cifrado con AES-256-GCM. A cambio de seguridad, la base no puede buscar ni ordenar usando estas, porque solo ve texto

cifrado, por eso únicamente se cifran datos que nunca se van a utilizar para buscar, como teléfonos y notas. Los nombres y las matrículas, que si se buscan no se cifran, se protegen con los permisos por rol.

4. Diagrama entidad-relación

Este es el diagrama de entidad-relación a partir de la base de datos real utilizada para SIRAI. Se muestran todas las tablas con su tipo exacto y relación.

Símbolo	Significado
Llave amarilla	Llave primaria: identifica a cada registro.
Rombo Rojo	Llave foránea: apunta a otra tabla.
Rombo (relleno azul o rojo)	La columna es obligatoria (NOT NULL).
Rombo vacío	La columna puede quedar vacía (NULL).
Línea continua	La tabla hija necesita a la padre para existir; la llave foránea forma parte de su llave primaria.
Línea punteada	La tabla hija solo apunta a la padre; tiene su propio id.
| | en un extremo, "pata de gallo" en el otro.	"Uno" del lado de la barra, "muchos" del lado de la pata de gallo.



5. Infraestructura

Para este proyecto se midió el uso de memoria y poder de hardware que utilizaba nuestra plataforma, utilizando estas tecnologías y la base de datos. Para la medición, se realizó un proceso de encendido para las tres piezas importantes (backend, frontend y base de datos) midiendo cuánto procesador (CPU) y memoria (RAM) usaba cada uno en dos momentos diferentes.

- En reposo: 20 segundos sin que nadie usara el sistema.
- Bajo carga: 212 conexiones al mismo tiempo durante 30 segundos, pidiendo el pase de lista.

El número sale del SRS: una conexión por cada una de las 212 estancias.

La herramienta de carga fue autocannon, y el script que hace todo es backend/pruebas/medicion.js. El CPU se expresa como porcentaje de un solo núcleo: 100 % es un núcleo completo trabajando.

5.1 Resultados

Los resultados obtenidos fueron los siguientes:

CPU y RAM	Backend (Node + Express)	Frontend (Vite + React-admin)	MySQL 8
CPU reposo	0 %	0 %	~ 0.3 %
CPU bajo carga (med/max)	~ 27 % ~ 31 %	0 % 0 %	~ 62 % ~ 64 %
RAM reposo	~ 210 MB	~ 46 MB	~ 451 MB
RAM bajo carga (med/max)	~ 275 MB	~ 46 MB	~ 470 MB

Con esa carga de trabajo, el sistema atendió un promedio de 1050 peticiones por segundo, todas con un resultado correcto (sin errores ni tiempos agotados), con una respuesta típica de 0.2 segundos.

5.2 Interpretación de resultados

De las tres piezas claves, MySQL es el que llega a trabajar más, utilizando un 62% de un núcleo. Esto es lo esperado, ya que es aquí en donde está toda la información. Cada vez que alguien abre el pase de lista, MySQL hace un total de 8 consultas:

- **4 son del pase de lista:** Qué turno opera la estancia, si ya cerró, la lista de infantes con su asistencia, su candado de reingreso, entre otros procesos.
- **4 son de la bitácora:** Apartar la bitácora para que nadie más escriba al mismo tiempo, leer el último hash, guardar el registro nuevo y liberarla.

Con esto identificamos que la mitad del trabajo de MySQL es registrar quién lo llegó a realizar. Este es el costo que se llega a adquirir cuando se quiere tener una bitácora de auditoría de logs encadenada a todos los endpoints.

Lo que genera que MySQL no utilice aún más del núcleo son los índices. Esto facilita la consulta de información de las tablas sin la necesidad de revisar la tabla completa, agilizando y minimizando el costo del proceso.

El backend en Node usa poco del núcleo, con tan solo un 27%. Node ejecuta el código en un solo hilo, haciendo que no pase de un solo núcleo. Bajo esta carga, podemos identificar que se tiene casi 4 veces de margen antes de necesitar más copias del backend. Esta memoria sube y baja de (210 a 275 MB) porque Node tiene una propiedad de reservar la memoria mientras atiende y luega la libera. Para caulcaur el RAM se utilizo el pico y no el promedio de uso.

El frontend casi no consume en el servidor, utilizando únicamente un 0%. React-admin corre en el navegador de cada persona, no en el servidor. En desarrollo, Vite ocupa 46 MB. En producción ni siquiera hace falta un proceso de Node: npm run build genera archivos estáticos que cualquier servidor web entrega casi sin gasto.

5.3 Uso de disco

Un registro de la bitácora utiliza unos 550 bytes con sus respectivos índices. La base de datos completa tiene un uso de espacio de 67.5 MB, de los cuales son 127,041 registros.

La bitácora es el elemento que más llega a crecer, esto se debe a su propiedad de registrar cada petición y que llega a ser inmutable (no se puede borrar la información).

5.4 ¿Qué se necesita para el programa?

Para que sea posible poder correr el programa, es necesario tener los siguientes elementos:

Elemento	Mínimo	Recomendado	Explicación
CPU	2 núcleos	4 núcleos	Bajo carga se usan 0.9 núcleos en total. Con 2 núcleos ofrece un margen amplio.
RAM	2 GB	4 GB	Los picos suman 0.8 GB, más el sistema operativo y el margen.
Disco	20 GB SSD	50 GB SSD	Lo decide el crecimiento de la bitácora.
Red	HTTPS	HTTPS	Todo tiene que viajar cifrado por temas de seguridad.

5.4 Límites de la medición

Para este análisis se terminó utilizando una computadora Mac, con MySQL dentro de Docker. En un servidor real, estos números obtenidos pueden llegar a variar; por ende, se utilizaron los números como posibles estimaciones del uso de nuestra plataforma para espacio y tiempo. Solo se llegó a medir el pase de lista; los demás procesos no se llegaron a tomar en cuenta. Se realizó la prueba con pocos datos; por ende, las consultas fueron más rápidas, aunque la adición de los índices ayudó en esto para agilizar el proceso.

6. Diferencias de nuestro SIRAI y el SRS

Esta es la información que se pide en el SRS que no se implementó del todo en esta primera versión del SIRAI.

Lo que pide el SRS	Lo que se hizo	¿Por qué?
Particionamiento por periodo y retención a 5 años.	No se particionar las tablas.	Se resuelve al operar la base en producción, no en una demo con datos de prueba. Queda como recomendación: particionar “asistencias” y “audit_log” por año.
Límite de sesiones por estancia	Existen la tabla “refresh_tokens” y la columna “estancias.max_sesiones”, pero el sistema todavía no las usa.	El JWT token de 8 horas cubre la autenticación. Las tablas quedaron listas para hacerlo después.
Stack PHP / CodeIgniter 4	Node, Express, React-Admin y MySQL	Es el stack del curso, acordado con el Project Plan.
Modo sin conexión (sincronización)	No se hizo.	Demasiada complejidad para el tiempo del proyecto.